

Topic to be covered

- Numpy Array vs Python List
- Advance Indexing
- Fancy Indexing
- Boolean Indexing
- Broadcasting
- Broadcasting Rules
- Broadcasting Examples
- Mathematical operation in numpy
- sigmoid in numpy
- Mean Square error in numpy
- Working with missing values
- Plotting Graphs

1.Numpy Array vs Python List

```
In [1]: # speed
# list

# mai in dono list k upar item wise addition karunga
# pehle item utaunga pehle list ka aur pehla item utaunga dusre list ka dono k
# aur aisa pure ek crore items k liye karne wala hoon

a = [i for i in range(10000000)]
b = [i for i in range(10000000,20000000)]

c = []
import time

start = time.time()
for i in range(len(a)):
    c.append(a[i]+b[i])
print(time.time()-start)
```

6.160005331039429

```
In [2]: # numpy

import numpy as np

a = np.arange(10000000)
b = np.arange(10000000,20000000)

start = time.time()
c = a+b
print(time.time()-start)
```

0.33596205711364746

```
In [3]: 1.87/0.13
```

```
Out[3]: 14.384615384615385
```

```
In [4]: # memory
a = [i for i in range(10000000)]

import sys
sys.getsizeof(a)
```

```
Out[4]: 89095160
```

```
In [5]: a = np.arange(10000000)
sys.getsizeof(a)
```

```
Out[5]: 40000112
```

2.Advance Indexing

```
In [6]: # Normal Indexing

a = np.arange(24).reshape(6,4)
a
```

```
Out[6]: array([[ 0,  1,  2,  3],
               [ 4,  5,  6,  7],
               [ 8,  9, 10, 11],
               [12, 13, 14, 15],
               [16, 17, 18, 19],
               [20, 21, 22, 23]])
```

```
In [7]: a[1,2]
```

```
Out[7]: 6
```

```
In [8]: a[1:3,1:3]
```

```
Out[8]: array([[ 5,  6],
               [ 9, 10]])
```

```
In [9]: # Fancy Indexing

# me kya karte ho aap same square bracket k under ek list pass kar dete ho
# aur list me jo chahiye hota hai aap uska index position de dete ho

# man lo hame 1st,2nd aur last row nikalna hai
a[[0,1,-1]]
```

```
Out[9]: array([[ 0,  1,  2,  3],
               [ 4,  5,  6,  7],
               [20, 21, 22, 23]])
```

```
In [10]: # man lo hame 1st,2nd aur 3rd column nikalna hai
a[:,[0,2,3]]
```

```
Out[10]: array([[ 0,  2,  3],
 [ 4,  6,  7],
 [ 8, 10, 11],
 [12, 14, 15],
 [16, 18, 19],
 [20, 22, 23]])
```

```
In [11]: # Boolean Indexing

# Boolean indexing tab use hota hai
# jab indexing ka koi concept nhi hai apko logic likh k uss number ko bahar ni
# eg:-
# aap puch sakte ho given array me se wo item nikal k do jo even hai
# ya aap mujhe wo no nikal k do joo five se divisible hai
# iss tarah k situation me aap jo indexing karte ho uss kehte hai boolean index
```

```
In [12]: a1 = np.random.randint(1,100,24).reshape(6,4)
a1
```

```
Out[12]: array([[41, 18, 30, 41],
 [54, 59, 50, 64],
 [51, 53, 16,  4],
 [ 7, 36, 10, 52],
 [23, 81, 23, 69],
 [11, 68, 90, 92]])
```

```
In [13]: # Find all numbers greater than 50

a1>50
```

```
Out[13]: array([[False, False, False, False],
 [ True,  True, False,  True],
 [ True,  True, False, False],
 [False, False, False,  True],
 [False,  True, False,  True],
 [False,  True,  True,  True]])
```

```
In [14]: a1[a1>50]
```

```
Out[14]: array([54, 59, 64, 51, 53, 52, 81, 69, 68, 90, 92])
```

```
In [15]: # find out even numbers

a1 % 2 == 0
```

```
Out[15]: array([[False,  True,  True, False],
 [ True, False,  True,  True],
 [False, False,  True,  True],
 [False,  True,  True,  True],
 [False, False, False, False],
 [False,  True,  True,  True]])
```

```
In [16]: a1[a1%2==0]
```

```
Out[16]: array([18, 30, 54, 50, 64, 16,  4, 36, 10, 52, 68, 90, 92])
```

```
In [17]: # find all number greater than 50 and are even
```

```
a1[(a1>50) & (a1 % 2 == 0)]
```

```
Out[17]: array([54, 64, 52, 68, 90, 92])
```

```
In [18]: # find all numbers not divisible by 7
```

```
a1[~(a1 % 7 == 0)]
```

```
Out[18]: array([41, 18, 30, 41, 54, 59, 50, 64, 51, 53, 16,  4, 36, 10, 52, 23, 81,
                23, 69, 11, 68, 90, 92])
```

Broadcasting

The term broadcasting describes how NumPy treats arrays with different shapes during arithmetic operations.

The smaller array is “broadcast” across the larger array so that they have compatible shapes.

```
In [19]: # Same shape
```

```
a = np.arange(6).reshape(2,3)
b = np.arange(6,12).reshape(2,3)

print(a)
print(b)

print(a+b)
```

```
[[0 1 2]
 [3 4 5]]
[[ 6  7  8]
 [ 9 10 11]]
[[ 6  8 10]
 [12 14 16]]
```

```
In [20]: # diff shape
```

```
a = np.arange(6).reshape(2,3)
b = np.arange(3).reshape(1,3)

print(a)
print(b)

print(a+b)
```

```
[[0 1 2]
 [3 4 5]]
[[0 1 2]
 [0 2 4]
 [3 5 7]]
```

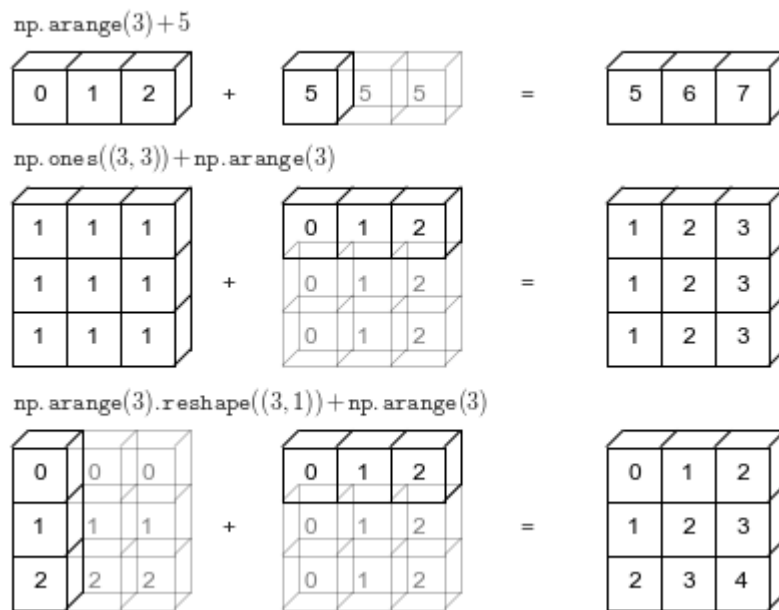
Broadcasting Rules

1. Make the two arrays have the same number of dimensions.

- If the numbers of dimensions of the two arrays are different, add new dimensions with size 1 to the head of the array with the smaller dimension.

2. Make each dimension of the two arrays the same size.

- If the sizes of each dimension of the two arrays do not match, dimensions with size 1 are stretched to the size of the other array.
- If there is a dimension whose size is not 1 in either of the two arrays, it cannot be broadcasted, and an error is raised.



In [21]: *# More examples*

```
a = np.arange(12).reshape(4,3)
b = np.arange(3)
```

```
print(a)
```

```
print(b)
```

```
print(a+b)
```

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
[0 1 2]
[[ 0  2  4]
 [ 3  5  7]
 [ 6  8 10]
 [ 9 11 13]]
```

```
In [22]: a = np.arange(12).reshape(3,4)
b = np.arange(3)

print(a)
print(b)

print(a+b)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[0 1 2]
```

ValueError

Traceback (most recent call last)

Cell In[22], line 7

```
      4 print(a)
      5 print(b)
----> 7 print(a+b)
```

ValueError: operands could not be broadcast together with shapes (3,4) (3,)

```
In [ ]: a = np.arange(3).reshape(1,3)
b = np.arange(3).reshape(3,1)

print(a)
print(b)

print(a+b)
```

```
In [ ]: a = np.arange(3).reshape(1,3)
b = np.arange(4).reshape(4,1)

print(a)
print(b)

print(a + b)
```

```
In [ ]: a = np.array([1])
# shape -> (1,1)
b = np.arange(4).reshape(2,2)
# shape -> (2,2)

print(a)
print(b)

print(a+b)
```

```
In [ ]: a = np.arange(12).reshape(3,4)
        b = np.arange(12).reshape(4,3)

        print(a)
        print(b)

        print(a+b)
```

Working with mathematical formula

```
In [ ]: a = np.arange(10)
        np.sin(a)
```

```
In [ ]: # sigmoid
        def sigmoid(array):
            return 1/(1 + np.exp(-(array)))

        a = np.arange(10)

        sigmoid(a)
```

```
In [23]: # mean squared error

        actual = np.random.randint(1,50,25)
        pred = np.random.randint(1,50,25)
```

```
In [25]: def mse(actual,pred):
            return np.mean((actual-pred)**2)

        mse(actual,pred)
```

Out[25]: 475.84

```
In [26]: actual
```

Out[26]: array([41, 2, 26, 23, 10, 32, 6, 42, 13, 41, 41, 26, 28, 13, 12, 18, 32,
 18, 42, 5, 23, 31, 6, 11, 3])

```
In [ ]: # categorical cross entrop
```

Working with missing value

```
In [27]: # working with missing value -> np.nan

        a = np.array([1,2,3,4,np.nan,6])
        a
```

Out[27]: array([1., 2., 3., 4., nan, 6.])

```
In [29]: a[~np.isnan(a)]
```

Out[29]: array([1., 2., 3., 4., 6.])

Plotting Graph

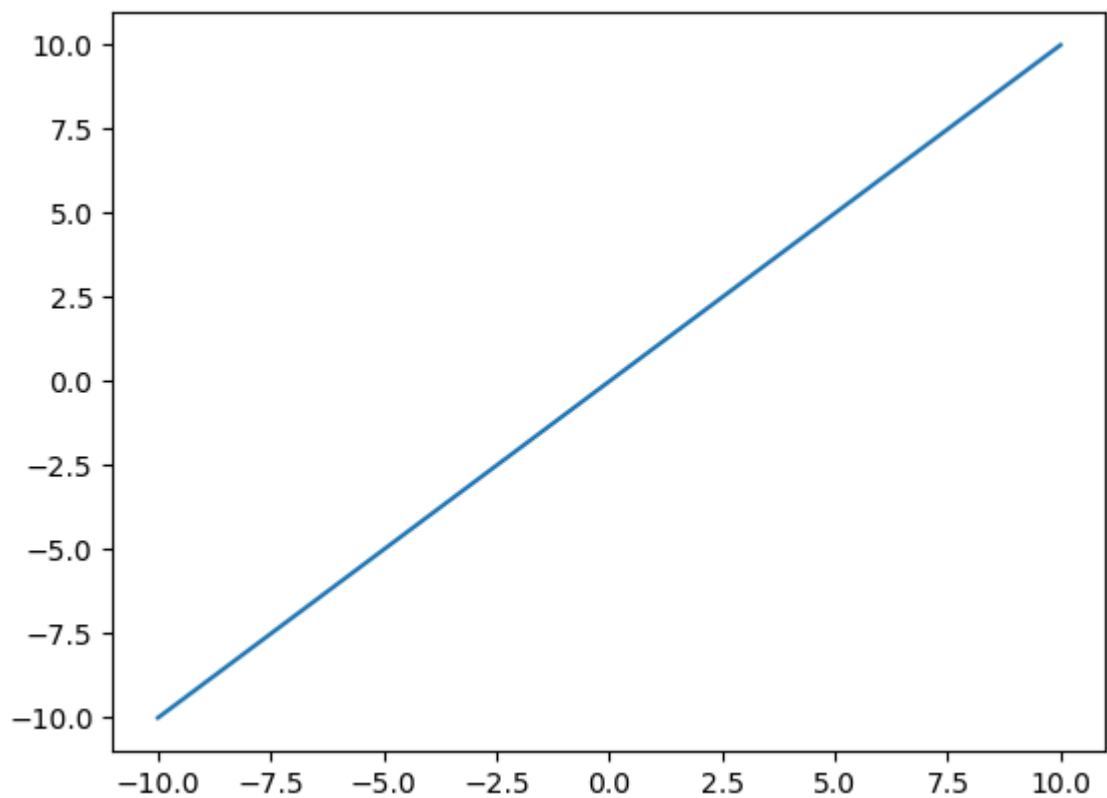
```
In [30]: # Plotting a 2d plot
# x = y

import matplotlib.pyplot as plt

x = np.linspace(-10,10,50)
y = x

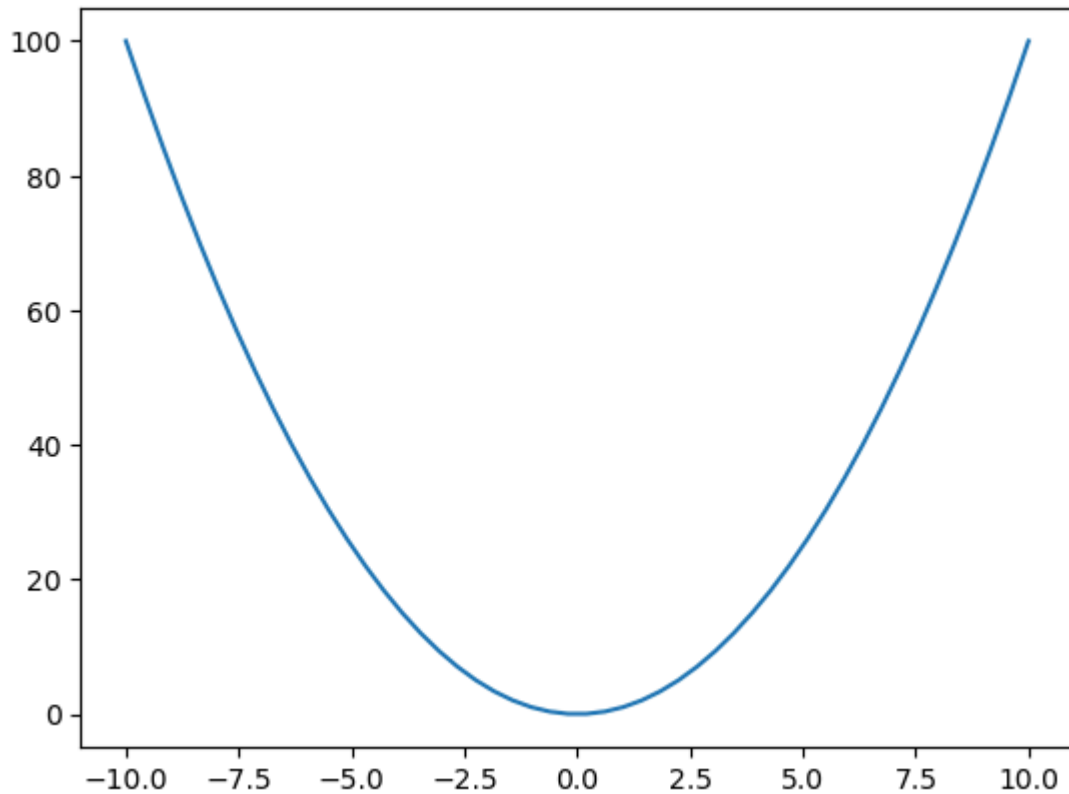
plt.plot(x,y)
```

Out[30]: [




```
In [31]: # y = x^2  
  
x = np.linspace(-10,10,50)  
y = x**2  
  
plt.plot(x,y)
```

Out[31]: [<matplotlib.lines.Line2D at 0x18002967f10>]



```
In [ ]: # y = sin(x)  
x = np.linspace(-10,10,100)  
y = np.sin(x)  
  
plt.plot(x,y)
```